Rapport de Projet

Application Web E-commerce

Développée avec Laravel (MVC)

Module	Programmation Web 2
Framework	Laravel (PHP)
Architecture	MVC (Modèle-Vue-Contrôleur)
Année académique	2025- 2026

Université — ESSECT

REALISE PAR :
YATHREB DELLAI
ISLEM NEFZI
MOLKA AMOR
ZEINEB MABROUK

Table des Matières

Table des Matières 2
1. Contexte du Projet 3
2. Objectifs de l'Application 3
3. Contraintes Techniques Respectées 3
4. Description des Tables 4
5. Implémentation Technique avec Laravel 5
5.1 Migrations 5
5.2 Eloquent ORM 5
5.3 Seeders & Factories 5
6. Intégrité et Sécurité de la Base de Données 5
7. Diagramme de Classes (UML) 5
8. Le Pattern MVC (Modèle-Vue-Contrôleur) 7
9. Flux de Fonctionnement — Cycle de Vie d'une Requête 7
10. Sécurité de l'Architecture 7
11. Diagramme des Cas d'Utilisation (UML) 8

PARTIE I — DESCRIPTION DU PROJET

1. Contexte du Projet

Ce projet est réalisé dans le cadre du module Programmation Web 2. L'objectif est de concevoir et de développer une application web d'e-commerce complète et dynamique en utilisant exclusivement le framework Laravel. Ce projet vise à démontrer notre maîtrise de l'architecture MVC, de l'ORM Eloquent, ainsi que de la sécurisation des échanges de données dans un contexte professionnel et réaliste.

2. Objectifs de l'Application

L'application permet la mise en relation entre vendeurs et acheteurs à travers une plateforme interactive et intuitive. Elle repose sur quatre piliers fonctionnels majeurs :

Authentification & Profil	Système sécurisé pour l'inscription, la connexion, la vérification d'email et la gestion des informations personnelles des utilisateurs.
Gestion du Catalogue	Ajout, modification et suppression de produits (titre, description, prix, catégorie, image). Le catalogue est public et offre des fonctionnalités de recherche, de filtrage et de tri.
Expérience d'Achat	Panier complet avec gestion des quantités et validation de commande assortie d'un suivi de statut (En attente, Validée, Annulée).
Évaluations	Module permettant aux clients de noter les produits et de laisser des commentaires afin d'instaurer une confiance au sein de la plateforme.

3. Contraintes Techniques Respectées

Conformément aux exigences du cahier des charges, l'application intègre les mesures techniques suivantes :

- Validation des formulaires pour garantir l'intégrité des données saisies par les utilisateurs.
- Sécurité renforcée contre les injections SQL, les failles XSS (Cross-Site Scripting) et CSRF (Cross-Site Request Forgery).
- Base de données structurée autour de 13 tables : users, products, categories, orders, order_items, reviews, cart_items, wishlists, sessions, cache, cache_locks, password_reset_tokens et migrations.
- Utilisation des Migrations et Seeders pour la génération automatique et reproductible de la structure de la base de données et des données de test.

4. Description des Tables

Notre base de données est structurée autour de 13 tables au total, couvrant l'ensemble des besoins fonctionnels de la plateforme e-commerce. Le tableau ci-dessous présente chaque table avec son rôle, son type de moteur de stockage ainsi que le nombre de lignes observées en phase de test :

Table	Description
`cache`	Stocke les données en cache applicatif pour améliorer les performances de l'application.
`cache_locks`	Gère les verrous sur le cache afin d'éviter les conditions de course lors des accès concurrents.
`cart_items`	Contient les articles ajoutés au panier par les utilisateurs, avant validation de la commande.
`categories`	Organise les produits par thématique pour faciliter la navigation et le filtrage du catalogue.
`migrations`	Historique des migrations Laravel, permettant la versionnisation et la portabilité de la base de données.
`orders`	Enregistre chaque commande globale : montant total, statut (En attente / Validée / Annulée) et référence utilisateur.
`order_items`	Table de liaison détaillant chaque article d'une commande : produit, quantité et prix unitaire au moment de l'achat.
`password_reset_tokens`	Stocke les tokens temporaires générés lors d'une demande de réinitialisation de mot de passe.
`products`	Contient les détails des articles : titre, description, prix, image et catégorie associée.
`reviews`	Stocke les notes (1 à 5 étoiles) et commentaires laissés par les utilisateurs sur les produits achetés.
`sessions`	Gère les sessions utilisateurs actives, assurant la persistance de l'état de connexion.
`users`	Stocke les informations d'authentification, les données personnelles et les rôles (Client / Admin).
`wishlists`	Permet aux utilisateurs de sauvegarder des produits dans une liste de favoris pour un achat ultérieur.

5. Implémentation Technique avec Laravel

5.1 Migrations

Chaque table a été créée via un fichier de migration dédié, assurant une versionnisation complète et une portabilité totale de la base de données entre les environnements de développement, de test et de production.

5.2 Eloquent ORM

Nous avons défini les relations logiques directement dans les modèles Eloquent (hasMany, belongsTo, belongsToMany), permettant de manipuler les données de manière sécurisée, lisible et maintenable, sans écrire de requêtes SQL brutes.

5.3 Seeders & Factories

Afin de valider l'application dans des conditions réalistes, des Seeders couplés à des Factories ont été mis en place pour générer automatiquement un jeu de données de test : catalogue de produits, utilisateurs fictifs, commandes simulées et avis.

6. Intégrité et Sécurité de la Base de Données

Contraintes d'intégrité référentielle

L'utilisation de clés étrangères (Foreign Keys) assure que la suppression d'une catégorie ou d'un utilisateur est gérée proprement. Par exemple, la directive onDelete('cascade') supprime automatiquement les enregistrements dépendants, évitant toute incohérence dans les données.

Précision monétaire

Pour les colonnes de prix (products.price, order_items.unit_price, orders.total_price), le type decimal(8,2) a été utilisé à la place du type float, afin d'éviter les erreurs d'arrondi lors des calculs de totaux de commande — un impératif dans tout contexte financier.

7. Diagramme de Classes (UML)

Le diagramme ci-dessous représente la structure de la base de données et les relations entre les 6 entités principales de l'application. Il constitue la fondation de notre architecture de données.

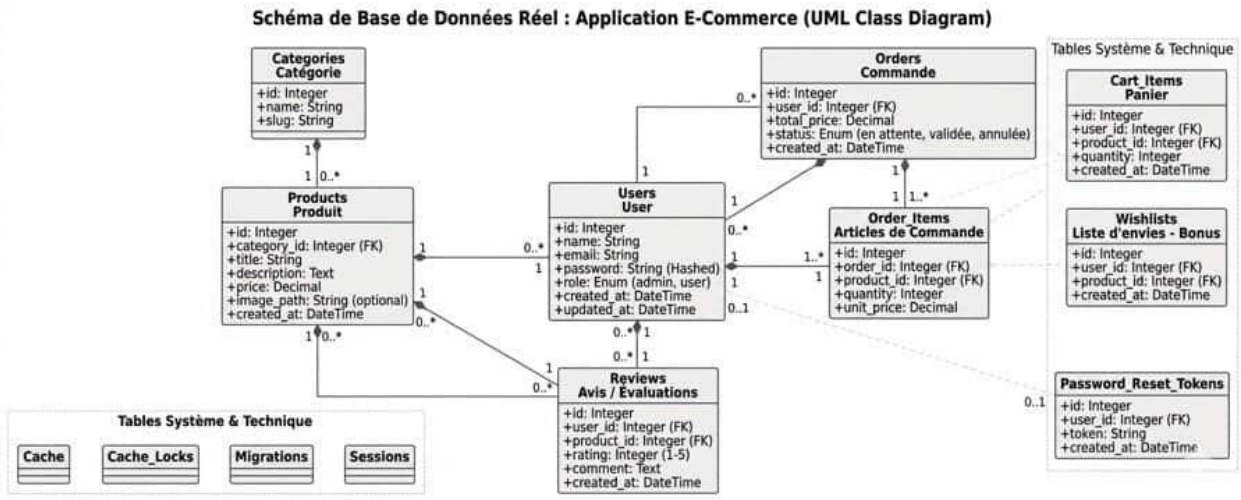

Figure 1 — Diagramme de Classes UML de l'application e-commerce

PARTIE II — ARCHITECTURE DU PROJET

8. Le Pattern MVC (Modèle-Vue-Contrôleur)

Notre projet repose sur l'architecture standard de Laravel, qui sépare clairement la logique métier de la couche d'affichage. Cette structure modulaire a permis à notre équipe de travailler simultanément sur différents composants sans créer de conflits majeurs dans le code source.

Modèle (Model)	Gère la logique des données et les interactions avec la base de données via Eloquent ORM. Chaque table possède son modèle correspondant qui définit les relations (ex. : un Product appartient à une Category via belongsTo).
Vue (View)	Gère l'interface utilisateur grâce au moteur de templates Blade, permettant de créer des mises en page (layouts) réutilisables et d'afficher dynamiquement les données de manière sécurisée.
Contrôleur (Controller)	Sert de pont entre le Modèle et la Vue. Il reçoit les requêtes HTTP (via les routes), interroge le Modèle pour récupérer les données nécessaires, et retourne la Vue appropriée enrichie de ces informations.

9. Flux de Fonctionnement — Cycle de Vie d'une Requête

Lorsqu'un utilisateur interagit avec la plateforme (exemple : ajouter un produit au panier), la requête suit le cycle technique ci-dessous :

1. Routage : La requête HTTP est interceptée par le fichier web.php, qui la route vers la méthode appropriée du contrôleur correspondant.
2. Middleware : Avant d'atteindre le contrôleur, la requête passe par des filtres de sécurité (ex. : le middleware auth vérifie si l'utilisateur est authentifié avant d'autoriser une commande).
3. Traitement : Le contrôleur exécute la logique métier (ex. : Cart::add($product)), met à jour la base de données via le Modèle Eloquent, puis prépare la réponse.
4. Rendu : La vue Blade est compilée en HTML pur et transmise au navigateur de l'utilisateur pour affichage.

10. Sécurité de l'Architecture

L'architecture Laravel offre des protections natives que nous avons intégralement exploitées :

- Protection CSRF : Utilisation de la directive @csrf dans tous nos formulaires pour prévenir les soumissions malveillantes provenant de sites tiers.
- Échappement automatique des données : Blade utilise la syntaxe {{ $variable }} pour encoder les données avant affichage, neutralisant ainsi les attaques XSS (Cross-Site Scripting).
- Validation centralisée : La validation des données est opérée directement dans les contrôleurs ou dans des Form Requests dédiés, garantissant que seules des informations valides (prix positif, email valide, etc.) entrent dans le système.
- Protection contre les injections SQL : L'utilisation exclusive d'Eloquent ORM et du Query Builder paramétré de Laravel empêche toute injection SQL directe.

11. Diagramme des Cas d'Utilisation (UML)

Le diagramme suivant illustre l'ensemble des fonctionnalités accessibles aux trois types d'acteurs de la plateforme : le Visiteur non connecté, l'Utilisateur connecté et l'Administrateur.

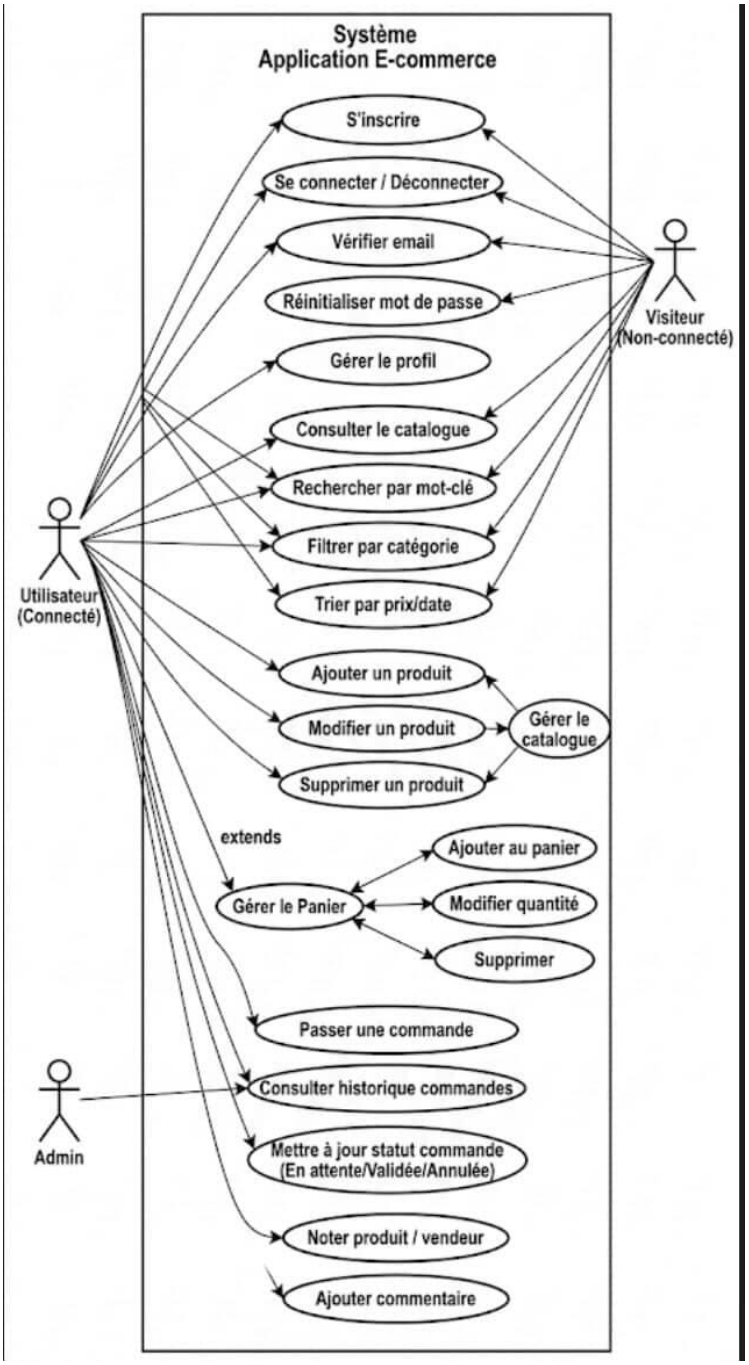

Figure 2 — Diagramme des Cas d'Utilisation UML de l'application e-commerce

CONCLUSION

Ce projet nous a permis de mettre en pratique l'ensemble des compétences acquises au cours du module Programmation Web 2. La conception d'une application e-commerce fonctionnelle avec Laravel nous a confrontés aux défis réels du développement web professionnel : structuration d'une base de données normalisée, sécurisation des échanges, architecture MVC rigoureuse et travail collaboratif en équipe.

L'utilisation du framework Laravel s'est révélée particulièrement efficace grâce à ses outils natifs (Eloquent ORM, Blade, Migrations, Middleware), qui nous ont permis de produire un code propre, maintenable et sécurisé. Ce projet constitue une base solide, extensible vers des fonctionnalités plus avancées telles qu'un système de paiement en ligne, des notifications en temps réel ou une API RESTful pour une application mobile complémentaire.